

DSA521S-GROUP MINI-PROJECT 2026

Title :NUST Service Centre
Simulation

GROUP MEMBERS;

225163993	Nantinda
226081745	Nashipili
225020238	Shilongo L.I
225149540	Jeremia Kambonde

PART A — DATA STRUCTURE DESIGN

TaskA1 — Waiting Line: Queue [6]

A queue to manage students waiting for service

Queue Diagram

State after 6 students arrive(enqueue):

Front

Rear

[Maria] -> [Tomas] -> [Ndapewa] -> [Simon] -> [Anna] -> [John]

State after 3 students are served (Dequeue):

Front

Rear

[Simon] -> [Anna] [John]

Pseudocode

```
class Queue{ static int size=6
    static int rear=-1 static int
    front=-1 static int[] queue =
    new int[5]
```

```
enqueue(student){
if (rear == size -1){
    DISPLAY "Queue is full"

else if(front == -1 && rear ==
-1 ){ front = 0 rear = 0
queue[rear] = student
```

```
rear ++queue[rear] =
student
```

```
dequeue(){ if(front -1
&& rear == -1){
    DISPLAY "Queue is empty"
else if(front ==
rear){ front = -1rear
= -
```

```
DISPLAY "Deleted element is: " + queue[front]
front ++
```

```
display(){ if(front == -1
&& rear == -1) DISPLAY
"Queue is empty"
```

```
for( i = front; i < rear + 1; i++){
DISPLAY queue[i]
```

```
peek(){ if(front == -1 &&
rear == -1){ DISPLAY
"queue is empty"
```

```
DISPLAY queue[front]
```

```
class Main {
Queue.enqueue(student)
Queue.enqueue(student)
Queue.enqueue(student)
Queue.enqueue(student)
Queue.enqueue(student) Queue.enqueue(student)
Queue.dequeue()
Queue.dequeue()
Queue.dequeue()
Queue.display();
```

Explanation: Why a Queue is Appropriate

- A queue follows the First-In, First-Out (FIFO) principle.

- In a service centre scenario, students must normally be assisted in the exact order in which they arrive.
- The first student to arrive (e.g., Maria) should be the first one to be served, making a queue data structure ideal for managing this waiting line.

Task A2 — Student Service Records: Singly Linked List

Linked List Diagram (Before and After Operations)

Operation 1: Insertion at the Beginning (Inserting Maria)

Before (List with Tomas and Ndapewa):

[Head] -> [Tomas] -> [Ndapewa] -> null

After (Inserting Maria at the beginning): [Head]
-> [Maria] -> [Tomas] -> [Ndapewa] -> null

Operation 2: Deletion (Deleting Tomas)

Before: [Head] -> [Maria] -> [Tomas] -> [Ndapewa] -> null

After (Unlinking Tomas by changing Maria's next pointer to Ndapewa): [Head] -> [Maria] -> [Ndapewa] -> null

PSEUDOCODE

Insertion at the beginning:

```
Method          enqueue(student)
newNode = new Node(student)
if rear == null
    then front =
    newNode rear =
    newNode
```

```

    rear.next      =
newNode    rear    =
newNode end if

```

DELETION:

```

Method dequeue() if front
== null then print "Queue
is empty" return null end
if    servedStudent    =
front.data    front    =
front.next if front == null
then rear = null end if
return servedStudent

```

TaskA3— Postfix Expression Evaluation: Stack

stack-based algorithm to evaluate a postfix arithmetic expression Java

PSEUDOCODE

```

create empty stack tokens = split
expression by spaces

for each token in tokens:
    if token is empty:
        continue

    if token is "+" or "-" or "*" or "/" or "%":
        right = stack.pop()
        left = stack.pop()
        result = O

```

```
if token is "+": result = left + right
if token is "-": result = left - right if
token is "*". result = left * right if
token is "/" or "/": result = left / right
```

```
stack.push(result)
```

```
else:
```

```
stack.push(convert token to integer)
```

```
print "Current top: " + stack.peek()
```

```
return stack.pop()
```

```
function main():
```

```
expression = "5 3 + 2 *" print
```

```
"Expression: " + expression
```

```
result = evaluate(expression)
```

```
print "Final Result: " + result
```

Task A4— Daily Statistics: Array

PSEUDOCODE

```
serviceTimes = [12, 5, 8, 4, 15, 9]
```

```
totalStudents =
```

```
LENGTH(serviceTimes)
```

```
totalServiceTime = 0
```

```
highestServiceTime = serviceTimes[0]
```

```
lowestServiceTime = serviceTimes[0]
```

```
servicesLongerThanTen = 0
```

```

FOR (i=0 ; totalStudents- 1;i++) time =
    serviceTimes[i]    totalServiceTime    =
    totalServiceTime + time

IF (time    >    highestServiceTime    )THEN
    highestServiceTime = time
END IF

IF(    time    <    lowestServiceTime)    THEN
    lowestServiceTime = time
END IF

IF (time > 10) THEN servicesLongerThanTen =
    servicesLongerThanTen + 1
END IF
END FOR

IF( totalStudents > 0) THEN averageServiceTime =
    REAL(totalServiceTime) / totalStudents
ELSE
    averageServiceTime    =
    0.0
END IF

```

Task A5 — Data-Structure Justification

1.Queue (Task A1 —Waiting Line)

- Behavior: Operates on a First-In, First-Out (FIFO) principle.
- Justification: A service centre requires students to be assisted strictly in the chronological order of their arrival. The queue data structure mimics this real-world constraint perfectly through enqueue (joining the back of the line) and dequeue (serving from the front), ensuring complete fairness and sequence preservation.

2. Singly Linked List (Task A2 — Student Service Records)

- Behavior: Dynamic collection of nodes where each node contains data and a pointer to the next node.
- Justification: Student records fluctuate continuously throughout the day—new students register, and completed records are deleted. A singly linked list is ideal because it allows dynamic insertions and deletions at any point (beginning, middle, or end). The linked list's behavior allows for efficient dynamic memory allocation, meaning it can grow or shrink as students are added or removed.

3. Stack (Task A3 — Postfix Expression Evaluation)

- Behavior: Operates on a Last-In, First-Out (LIFO) principle.
- Justification: When an operator is encountered (e.g. +), the algorithm must retrieve the two most recently pushed numbers. The stack's push() and pop() operations perfectly model this by ensuring the most recently added operand is the first one accessed, allowing the correct pairing of operands and operators.

4. Array (Task A4 — Daily Statistics)

- Behavior: A fixed-size, contiguous block of indexed elements supporting direct sequential access.
- Justification: At the end of the simulated day, the total set of recorded service times is fully known and static. An array is well-suited for this because a simple algorithmic loop can rapidly traverse the fixed dataset to accumulate totals, find bounds (highest/lowest), and compute averages.

PART B — SORTING ALGORITHM CHALLENGE

Task B1 — Selection Sort using the provided array: [17, 5, 23, 8, 14, 3, 11, 20, 6, 9].

1. Array States After the First 3 Passes

- Initial Array: [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]
- After Pass 1 : [3, 5, 23, 8, 14, 17, 11, 20, 6, 9] => (Minimum 3 swapped with 17)
- After Pass 2: [3, 5, 23, 8, 14, 17, 11, 20, 6, 9] => (Minimum 5 is already in place; no swap needed)
- After Pass 3: [3, 5, 6, 8, 14, 17, 11, 20, 23, 9] => (Minimum 6 swapped with 23)

Task B2 — Insertion Sort using the service-time array: [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

1. Array States After the First 3 Passes

- Initial Array: [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]
- After Pass 1: [5, 17, 23, 8, 14, 3, 11, 20, 6, 9] => (Element 5 inserted before 17)
- After Pass 2: [5, 17, 23, 8, 14, 3, 11, 20, 6, 9] => (Element 23 is already in its correct sorted position relative to the left)
- After Pass 3: [5, 8, 17, 23, 14, 3, 11, 20, 6, 9] => (Element 8 inserted between 5 and 17 after shifting 17 and 23)

Task B3— Merge Sort

[17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

[17, 5, 23, 8, 14] [3, 11, 20, 6, 9]

[17, 5] [23, 8, 14] [3, 11] [20, 6, 9]

[17] [5] [23] [8, 14] [3] [11] [20] [6, 9]

[8] [14]

BASE CASE:

[1 7] [5] [23] [8] 4 [3] [1 1] [20] [6] [9]

MERGE BACK:

1[5] [23] [8] [14] [3][11] [20] [6] [9]

[5, 1 7] [23] [8, 14] [3, 1 1] [20] [6, 9]

∖ [8, 14, 23] ∖ [6, 9, 20]

[5, 8, 14, 1 7, 23] [3, 6, 9, 1 1, 20]

[3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

Task B4 — Quick Sort

Pivot-Selection Rule

- Rule Used: Choose the last element of the current sub-array as the pivot (pivot = arr[high]).

First Two Partitioning Stages

Stage 1 (Entire Array)

- Initial Array: [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]
- Pivot: 9 (at index 9)
- Partitioning Process: Rearranging elements so that elements smaller than or equal to 9 are on the left, and elements greater than 9 are on the right.
- Result of Stage 1 Partition:
- Left Partition: [5, 8, 3, 6] o Pivot in place: 9 o Right Partition: [23, 11, 20, 17, 14]
- Arraystate afterstage 1: [5, 8, 3, 6, 9, 23, 11, 20, 17, 14]

Stage 2 (Partitioning the Left Sub-Array)

- Sub-Array: [5, 8, 3, 6]
- Pivot: 6 (the last element of this sub-array)
- Partitioning Process: Rearranging elements relative to pivot 6.
- Result of Stage 2 Partition:
- Left Sub-Partition: [5, 3] o Pivot in place: 6 o Right Sub-Partition: [8]
- Arraystate after Stage 2: [5, 3, 6, 8, 9, 23, 11, 20, 17, 14]

PART C — ALGORITHM EXPERIMENT

PSEUDOCODE

SortingExperiment

SET sizes = [20, 50, 100, 500]

FOR each size IN sizes:

 Generate random array of given size

 CALL SelectionSort(array), print results

 CALL InsertionSort(array), print results

```
CALL MergeSort(array), print results
CALL QuickSort(array), print results
END FOR
PRINT header for Almost-Sorted Test
// Almost-Sorted Test (Size 100)
Generate sorted array of size 100
Swap 5 specific adjacent pairs to slightly disorder the array
CALL all 4 sorts on this array, print results
```

1 . SELECTION SORT

```
SelectionSort(original):
  CLONE original into arr
  SET comparisons = 0
  START timer

  FOR( i = 0   arr.length - 2;i++)
    SET minIndex = i
    FOR j = i + 1 to arr.length - 1 :
      INCREMENT comparisons
      IF arr[j] < arr[minIndex]:
        SET minIndex = j
      END IF
    END FOR
    SWAP arr[i] and arr[minIndex]
  END FOR

  STOP timer

  RETURN comparisons and elapsed time
```

2. INSERTION SORT

InsertionSort(original):

CLONE original into arr

SET comparisons = 0

START timer

FOR (i = 1 ;i<=to arr.length- 1;i++)

SET key = arr[i]

SET j= i - 1

WHILE j > 0:

INCREMENT

comparisons IF arr[j] >

key: SHIFT arr[j] to arro

+ 1]

DECREMENT j

ELSE:

BREAK

END IF

END WHILE

SET arr[j + 1] = key

END FOR

STOP timer RETURN comparisons

and elapsed time

3. MERGE SORT

MergeSort(original):

CLONE original into arr

SET comparisons = 0

START timer

CALL RecursiveSort(arr, 0, arr.length - 1)

STOP timer

RETURN comparisons and elapsed time

FUNCTION RecursiveSort(arr, left, right):

IF(left >= right)

RETURN

END IF

SET mid = left + (right - left) / 2

CALL RecursiveSort(arr, left, mid)

CALL RecursiveSort(arr, mid + 1, right)

CALL Merge(arr, left, mid, right)

Merge(arr, left, mid, right):

Create temp arrays L and R from arr

SET i = 0, j = 0, k = left

WHILE (i < L.length AND j < R.length)

INCREMENT comparisons

IF L[i] <= R[j]:

arr[k++] = L[i++]

ELSE:

arr[k++] = R[j++]

END IF

END WHILE

COPY remaining elements of L and R into arr

4. QUICK SORT

QuickSort(original)

CLONE original into

arr

SET comparisons = 0

START timer

CALL RecursiveQSort(arr, 0, arr.length - 1)

STOP timer

RETURN comparisons and elapsed time

RecursiveQSort(arr, low, high):

IF (low < high)

SET pivotIndex = Partition(arr, low, high)

CALL RecursiveQSort(arr, low, pivotIndex - 1)

CALL RecursiveQSort(arr, pivotIndex + 1, high)

END IF

Partition(arr, low, high):

SET pivot = arr[high]

SET i = low - 1

FOR(j = low to high - 1)

INCREMENT comparisons

IF arr[j] <= pivot

INCREMENT i

SWAP arr[i] and arr[j]

END IF
END FOR

SWAP arr[i + 1] and arr[high]
RETURN i + 1

Answers to Analysis Questions

1 . Which algorithm performed the fewest comparisons?

- Answer: Insertion Sort performed the fewest comparisons when tested on the almostsorted array, because elements were already nearly in place, requiring minimal inner loop checks.

2. Which algorithm performed the most comparisons?

- Answer: Selection Sort performed the most comparisons overall across random inputs.

3. Which algorithm performed best on the almost-sorted array?

- Answer: Insertion Sort. Because the array was nearly sorted.

4. Which algorithm appeared to scale best as the input became larger?

- Answer: Merge Sort and Quick Sort. As the input size scaled up from 100 to 500, their execution times and comparison counts grew at a linear rate whereas Selection and Insertion sorts spiked dramatically.

5. Did the experimental results generally agree with the theoretical time complexity? Explain?

- Answer: Yes. The results closely matched theory: Selection Sort scaled quadratically (n^2), Merge and Quick Sort scaled efficiently, and Insertion Sort adapted dynamically to input ordering (performing poorly on random data but exceptionally well on nearlysorted data).

6. Is execution time alone enough to decide which algorithm is better? Explain?

- Answer: No. Execution time can be heavily influenced by external factors such as JVM warm-up time, garbage collection pauses, hardware architecture (CPU caching), and compiler optimizations. Furthermore, an algorithm might have a fast execution time on average (like Quick Sort on random data) but have terrible worst-case behavior making it unsuitable for memory-constrained environments.

PART D — INTEGRATED SERVICE-CENTRE SYSTEM

The Integrated Service-Centre System combines the Queue, Singly Linked List, Array processing, and Sorting components into one Java program. The system uses a menu-driven interface that allows the user to manage students waiting for service, maintain student service records, calculate daily statistics, and sort service times.

System Menu

CAMPUS SERVICE CENTRE

- 1 . Add student to waiting queue
 2. Serve next student
 3. Display waiting students
 4. Add student service record
 5. Display student service records
 6. Search for student record
 7. Remove student record
 8. Display daily statistics
 9. Sort service times
 10. Run sorting experiment
 - 11 . Exit
-

Select option:

Explanation

The menu provides access to all the required data structures and algorithms.

Each menu option calls the appropriate operation rather than performing unrelated operations.

GitHub Link: <https://github.com/225149540-kamonde/DSA521S-GroupXX-Project2026>

The Integrated Service-Centre System successfully combines the required data structures and algorithms into one menu-driven Java program.

The Queue is used to manage students waiting for service, while the Singly Linked List stores and manages student service records. The Array stores service times and is used to calculate daily statistics. The implemented sorting algorithms are used to arrange the service-time data and conduct the sorting experiment.